

Design, analysis and experimental evaluation of randomized algorithms for the minimum cut problem

Marko Pongrac

Departamento de Eletrónica, Telecomunicações e Informática
mpong rac@ua.pt

Keywords – graph theory, non-deterministic algorithms, heuristics, minimum cut, optimization

Abstract – This report investigates randomized algorithms for finding the global minimum cut in undirected, unweighted graphs, focusing on random edge contractions and random partition initializations. While these heuristic approaches offer efficient and often accurate solutions, their success probability can be low for certain graph structures. To address this, several strategies are presented: fully randomized contraction method, hybrid method that combines random contractions with exhaustive search on reduced graphs, and heuristic approach based on multiple random initializations. The results show that while random algorithms with too small number of iterations fail on larger graphs, hybrid algorithms that integrate randomness with structural knowledge consistently achieve higher precision and better coverage of the solution space, highlighting that controlled randomness paired with graph-aware heuristics provides an effective approach to the minimum cut problem.

I. INTRODUCTION

As previously discussed in [1], the minimum cut of the graph G is a partition of its vertices into two disjoint subsets S and T such that the number of edges connecting the two subsets is minimal, i.e., the size of the cut of the graph is defined as

$$c(S, T) = |\{ \{u, v\} \in E \mid u \in S, v \in T \}|.$$

While the minimum cut is such a cut that satisfies the condition

$$c(S^*, T^*) = \min_{(S, T)} c(S, T).$$

It is well known that exhaustive approaches to this problem suffer from the combinatorial explosion, since the total number of configurations to test grows exponentially with either the number of vertices or the number of edges in the given graph.

Unlike the exhaustive algorithms, greedy algorithms for the minimum cut problem usually work in a way that they either iteratively move vertices between the subsets S and T , in order to reduce the current cut size, or they iteratively remove edges, in order to achieve the

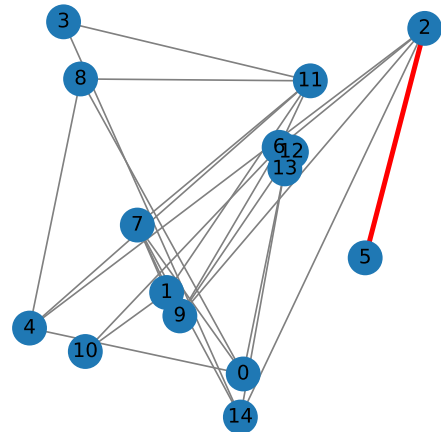


Fig. 1 - Minimum cut of graph with 15 vertices and 27 edges. The only edge that belongs to the minimum cut of the size 1 is the edge between vertices 2 and 5 (highlighted in red).

disconnected graph. In both cases, decisions are made locally, by using the predetermined heuristic function. While these approaches are capable of running in polynomial time and also yield good results, they are also prone to getting stuck in local optimums or, in the case of minimum cut problem, local minimums. A local minimum is a partition configuration in which no single move sequence can further decrease the cut size, even though a globally smaller cut exists.

To overcome this difficulty, randomized strategies can be incorporated in the existing deterministic approaches, for example randomly initializing partitions or combining greedy updates with randomness, an idea widely used in genetic algorithms [2]. These slight modifications increase the likelihood of escaping the local optima and exploring bigger portion of solution space, therefore increasing the probability of finding the globally optimum solution, in our case global minimum cut.

Here it is important to note that randomized algorithms rely on stochastic decisions during the execution. Consequently they may not find the optimal solution in a single run. However, running the algorithm multiple times with independent random initializations significantly increases the probability of finding the minimum cut, as each run explores a different part of the solution space. Let p be the probability of

finding the optimal solution in a single run, and n be a number of independent runs. Then, the probability of finding the optimal solution is

$$P(\text{optimal solution found}) = 1 - (1 - p)^n,$$

which shows that subsequent independent runs significantly increase the probability of finding the optimal solution.

II. RANDOMIZED ALGORITHMS

A. Random Contractions

The idea behind this approach is to repeatedly contract randomly selected edges until only two *super vertices* remain. Each contraction merges the vertices of the selected edge into a single vertex. By doing this, the algorithm reduces the graph size while preserving the structural information relevant to the given problem. Once only 2 *super vertices* remain, the cut size is obtained by simply counting the remaining edges between the vertices. This algorithm is based on the algorithm already proposed by Karger [3].

Algorithm 1 Random Contractions for Minimum Cut

```

1: Input: Graph  $G = (V, E)$ , maximum iterations
2: Output: Minimum cut partition  $(S, T)$  and cut size
3:  $min\_cut\_size \leftarrow |E|$ 
4:  $min\_cut \leftarrow \text{None}$ 
5: for  $i = 1$  to  $max\_iter$  do
6:   Initialize  $components[v] \leftarrow \{v\}$  for all  $v \in V$ 
7:   while number of vertices  $> 2$  do
8:     Pick a random edge  $(u, v) \in edges$ 
9:     Merge vertex  $v$  into vertex  $u$ :
10:    Remove self-loops from  $u$ 
11:    Merge  $components[v]$  into  $components[u]$ 
12:    Delete  $v$  and its component
13:   end while
14:   Let remaining vertices be  $a$  and  $b$ 
15:    $cut\_size \leftarrow$  number of edges between  $a$  and  $b$ 
16:   Generate representations of  $[components[a], components[b]]$  and compute hash
17:   if hash not seen before then
18:     if  $cut\_size < min\_cut\_size$  then
19:        $min\_cut \leftarrow [components[a], components[b]]$ 
20:        $min\_cut\_size \leftarrow cut\_size$ 
21:     end if
22:   end if
23: end for
24: return  $min\_cut, min\_cut\_size$ 

```

The single run of this algorithm may not find the optimal solution, since random contraction may eliminate the edges that belong to the true minimum cut. Therefore, the algorithm is run multiple times, to increase the probability of finding the optimal solution. To ensure that the same solution is not

tested more than once, hash value of the canonical representations of the obtained partitions $cut_hash = \text{hashlib.sha256}(\text{str}(\text{canon}).\text{encode}()).\text{hexdigest}()$ is calculated and stored. In case that the found solution has size 1, early stopping is performed, since that is the lower bound for minimum cut size of the connected graphs.

B. Contraction-Then-Exhaustive

While previously described algorithm is efficient, it is also completely randomized, therefore potentially resulting in suboptimal minimum cuts. To improve the reliability and precision of the algorithm, contractions are performed only until reaching the certain size of the graph, after which the exhaustive partition enumeration follows. Therefore, for graphs with 7 vertices or less contraction step is completely skipped, while exhaustive search never checks more than $2^6 - 1$, i.e. 63 bipartitions. Once again, hash value of the contracted graph is calculated and stored, in order to skip exhaustive search on the already tested configurations. Early stopping is performed in case that the found cut has size 1.

Algorithm 2 Random Contractions with Exhaustive Search for Minimum Cut

```

1: Input: Graph  $G = (V, E)$ , max iterations
2: Output: Minimum cut  $(S, T)$  and cut size
3:  $best\_cut\_size \leftarrow \infty$ ,  $best\_cut \leftarrow \text{None}$ ,  $seen\_hashes \leftarrow \emptyset$ 
4: for  $i = 1$  to  $max\_iter$  do
5:    $contracted, components \leftarrow \text{RandomContractionUntil7}(G)$ 
6:    $cut\_hash \leftarrow \text{hash}(\text{sorted lists of vertices in } components)$ 
7:   if  $cut\_hash \in seen\_hashes$  then continue
8:   end if
9:   add  $cut\_hash$  to  $seen\_hashes$ 
10:   $partitions \leftarrow \text{GenerateNodePartitions}(\text{vertices in } contracted)$ 
11:   $min\_local \leftarrow \infty$ ,  $best\_S, best\_T \leftarrow \emptyset$ 
12:  for all  $(S, T) \in partitions$  do
13:     $current\_cut \leftarrow$  number of edges between  $S$  and  $T$  in  $G$  using  $components$ 
14:    if  $current\_cut < min\_local$  then
15:       $min\_local \leftarrow current\_cut$ ,  $best\_S, best\_T \leftarrow S, T$ 
16:    end if
17:    if  $current\_cut == 1$  then break
18:    end if
19:  end for
20:  if  $min\_local < best\_cut\_size$  then
21:     $best\_cut\_size \leftarrow min\_local$ ,  $best\_cut \leftarrow (best\_S, best\_T)$ 
22:  end if
23: end for
24: return  $best\_cut, best\_cut\_size$ 

```

This hybrid approach improves robustness by avoid-

ing repeated exploration of identical contracted graphs (via hashing) and by reducing the graph less aggressively than the standard **Random Contractions** algorithm. This allows more structural information of the original graph to be preserved while keeping the subsequent exhaustive set enumeration computationally feasible. As a result, the method maintains practical efficiency while guaranteeing an exact evaluation on the small contracted instances.

C. Greedy Randomized Partitions

The Greedy Randomized Partitions algorithm is a slight enhancement of the basic greedy partitioning approach, presented in [1], by introducing multiple runs starting from the random initial partitions. This increases the size of the explored solution space, while also keeping the sub-exponential running time, since the number of runs is a constant.

Algorithm 3 Greedy Randomized Minimum Cut Heuristic

```

1: Input: Graph  $G = (V, E)$ , number of runs  $N$ 
2: Output: Best cut found  $(P^*, C^*)$ 
3:  $C^* \leftarrow \infty$ ,  $P^* \leftarrow \text{None}$ 
4: for  $i = 1$  to  $N$  do
5:   Randomly shuffle vertices  $V$  and create initial
     partition  $P$ 
6:   Compute cut size of  $P$ 
7:   Compute  $D[v]$  for all vertices
8:   repeat
9:      $U \leftarrow$  set of unlocked vertices
10:    Initialize move sequence and cut sequence
11:    while  $U \neq \emptyset$  do
12:      Move  $v \in U$  with maximum  $D[v]$ 
13:      Update partition sizes,  $D$  of neighbours
14:      record current cut size
15:      Remove  $v$  from  $U$ 
16:    end while
17:    Identify moves giving minimal cut
18:    if partition already hashed then
19:      break
20:    end if
21:    if cut improved then
22:      Apply best prefix to  $P$ , recompute  $D[v]$ 
23:    end if
24:  until no improvement
25:   $C \leftarrow$  final cut size of  $P$ 
26:  if  $C < C^*$  then
27:     $C^* \leftarrow C$ ,  $P^* \leftarrow P$ 
28:  end if
29: end for
30: return  $(P^*, C^*)$ 

```

Since multiple independent runs are introduced, it is desirable to keep track of the previously tested configurations in order to avoid redundant evaluations and ensure that refinement of the current configurations is only applied to the genuinely new candidate solutions. This can be done efficiently by computing the

hash value of the proposed candidate solution. However, the way in which partitions are stored has to be slightly changed, since previously used data structure **list** is not hashable. To solve this, I propose using the tuple of size $|V|$, where the value 1 at the index i indicates that the vertex v_i belongs to the partition S , while 0 indicates that it belongs to T .

The computation of the hash value for each partition has the time complexity of $O(|V|)$, as the entire tuple has to be processed. Checking membership in the already-processed-set is $O(1)$ on average, since in python it is implemented as a hash table [4].

Therefore, this approach preserves a high degree of solution quality while maintaining efficiency, as it exploits both the structural information of the graph and the diversity introduced by random initializations. The algorithm is particularly effective when combined with hashing, ensuring that refinement efforts are focused only on genuinely new candidate partitions.

D. Formal Complexity Analysis

To better understand the computational requirements of the different minimum cut algorithms, their average-case and worst-case time complexities are shown in Table I.

For algorithms that utilize random contractions, the time complexity of a *single contraction run* is $O(|V|^2 + |E|)$. The initial setup in each contraction process has the time complexity of $O(|V| + |E|)$, as it involves copying vertices and edges into more convenient data structure. The main loop of the contraction process runs $|V| - 2$ times. Inside the loop, operations such as deleting a vertex require $O(1)$ time, while traversing and updating adjacency lists of neighbours contributes a total of $O(|V|^2 + |E|)$ across all iterations. Consequently, the overall complexity of one contraction run is dominated by $O(|V|^2 + |E|)$. Repeating the algorithm k times gives the total complexity of

$$O(k \cdot (|V|^2 + |E|)).$$

Unlike the ordinary Random Contraction algorithm, the hybrid approach of Random Contractions with Exhaustive Search perform five less iterations of contracting process, before performing the exhaustive partition enumeration process on the reduced graph. In order to avoid performing exhaustive search on the already tested configurations, hash set is used to store the hash values of each reduced graph. Calculating the hash value has the time complexity of $O(|V|)$, while checking whether the hash was already stored is performed in constant time $O(1)$. Exhaustive partition enumeration in each iteration tests at most $2^6 - 1$, or 63 partitions, while calculating the cut size of the current partition has complexity of $O(n^2)$. Therefore, the final expression of the for the time complexity of the single iteration is $O(|V|^2 + |E|)$, while the number of iterations is k .

The time complexity of the Greedy Partition Algorithm was already discussed in [1], where it has the

TABLE I
TIME COMPLEXITY ALGORITHMS

Algorithm	Average-case Time Complexity	Worst-case Time Complexity
Random Contractions	$\Theta(k \cdot (V ^2 + E))$	$O(k \cdot (V ^3))$
Hybrid method	$\Theta(k \cdot (V ^2 + E))$	$O(k \cdot (V ^3))$
Random partitions	$\Theta(k \cdot (V ^2 + E))$	$O(k \cdot V ^3)$

proposed average-case time complexity of $\Theta(k \cdot (|V|^2 + |E|))$, and the worst-case $O(k \cdot |V|^3)$, for very dense graphs.

III. EXPERIMENTAL METHODOLOGY

To evaluate the performance of the minimum cut algorithms, experiments were conducted on randomly generated undirected, unweighted graphs of varying sizes and densities. To enable comparison between exhaustive algorithm and different random approaches, the number of vertices ranged from 3 to 24. Edge densities were set to 0.125, 0.25, 0.5, 0.75, and 1, covering both sparse and dense graph topologies. Graphs with more than 7 vertices and edge probability set to lower than 1 were generated with 5 different topologies, giving in total 375 graphs. Additionally, 134 larger graphs containing up to 500 vertices and with different densities were created, in order to compare efficiency of greedy and random strategies. All algorithms were implemented in Python using the NetworkX python library [5] and the Python built-in library `hashlib` [6], and executed on an 11th Gen Intel(R) Core(TM) i5-11400H @ 2.70GHz processor. Every random algorithm had the number of iterations set to 100 and given a time limit of 5 seconds. Performance metrics include the size of the minimum cut found, execution time, number of basic operations performed, and number of configurations tested as possible solutions. Additionally, an important metric measured for random algorithm approaches was the precision metric, which measures how many times did the algorithm find the optimal solution, compared to the exhaustive method.

IV. RESULTS

A. Precision

The precision of the random algorithms was evaluated by comparing the results obtained with Exhaustive Partition Enumeration method, and then compared with the other random and greedy methods. Table II summarizes those results. Both of the algorithms that utilized random contraction approach, **Random Contractions** and **Random-Then-Exhaustive** algorithms achieved the perfect match compared to the exhaustive method, proving to be valid alternatives when looking for the more efficient solution.

The **Random Partitions** algorithm, while not achieving a perfect match like the fully randomized contraction approaches, still provides high precision, with a mean absolute error of 0.069 and exact matches in over 94% of the cases. This demonstrates that incor-

porating random initializations into greedy algorithms can slightly increase their precision, since the ordinary **Greedy Partitioning** achieves just over 93% score.

TABLE II
COMPARISON OF PRECISION OF MINIMUM CUT ALGORITHMS

Algorithm	MAE	MaxAE	Precision (%)
Greedy Edge Removal	0.0	0	100
Greedy Partitioning	0.085	3	93.1
Random Partitions	0.069	3	94.13
Random Contractions	0.000	0	100
Random-Then-Exhaustive	0.000	0	100

For the algorithms that manipulate partitions, the histogram of the frequencies of the non-zero absolute errors was shown in the figure 2, where we can see slight improvement achieved by introducing random initialization into greedy approach. Not only does the **Random Partitions** algorithm have lower frequency of misses by size 1, it also reduces the number of misses by size 2 in half, dropping from 4 for **Greedy Partitions** to 2 for **Random Partitions**. On the other hand, it seems that introducing randomness could not solve the example where it misses the minimum cut size by 3.

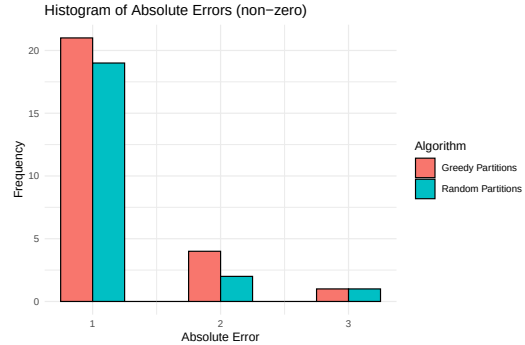


Fig. 2 - Histogram of the frequencies of non-zero absolute errors for the Random Partitions algorithm and Greedy Partitions.

B. Time Complexity

The relationship between the number of vertices of the input graph and the number of basic operations conducted is shown on the figures 3 and 4.

The two methods that utilize random contractions exhibit higher running time than the greedy approaches, **Greedy Partition** and **Greedy Edge Removal**, particularly on smaller graphs. However, the results show that the number of basic operations for both contraction-based algorithms grows sub-exponentially

with respect to the number of vertices, with the growth rate even slowing down as the graph size increases.

Similarly, the **Random Partitions** algorithm also demonstrates sub-exponential growth, although it starts with higher initial complexity compared to its purely heuristic variant. Overall, while the random-based methods are more expensive on small instances, they remain practical and clearly avoid combinatorial explosion across the tested graph sizes.

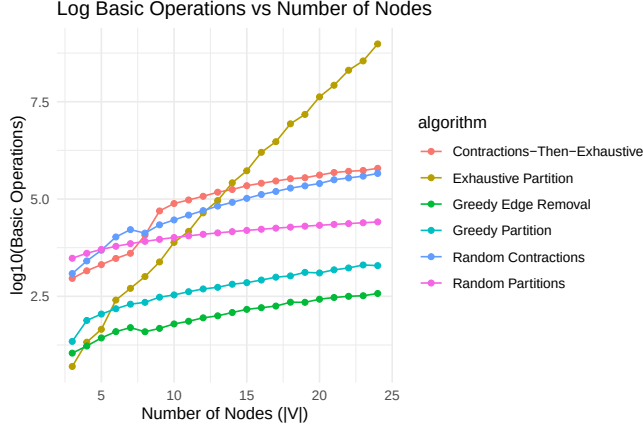


Fig. 3 - Logarithm of the average number of basic operations as a function of the number of vertices.

When examining the relationship between the logarithm of the average number of basic operations and the number of edges, all randomized methods consistently show sub-exponential time growth. This further supports the conclusion that their empirical time complexity remains well below exponential with respect to $|E|$, even though they are computationally heavier than the greedy heuristics.

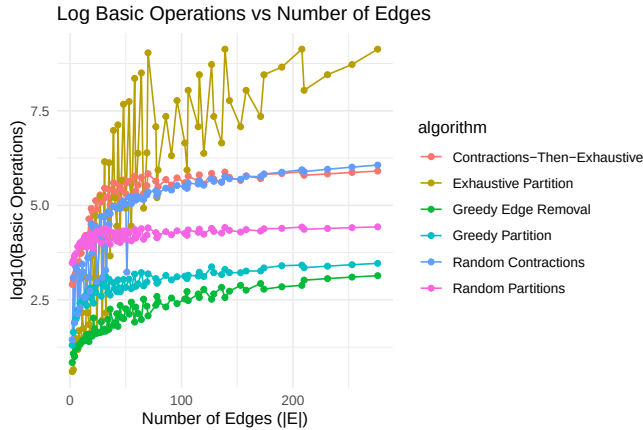


Fig. 4 - Logarithm of the average number of basic operations as a function of the number of edges.

C. Execution Time

As shown in Figure 5, the methods that incorporate randomness into their procedure again demonstrate sub-exponential growth in runtime as the number of vertices increases, confirming the earlier observations made for basic operation counts. Even though they

exhibit higher time complexity for smaller graphs than even **Exhaustive Partition**, they all start outperforming it once the number of vertices becomes 15 or more. Some methods, like **Random Partitions**

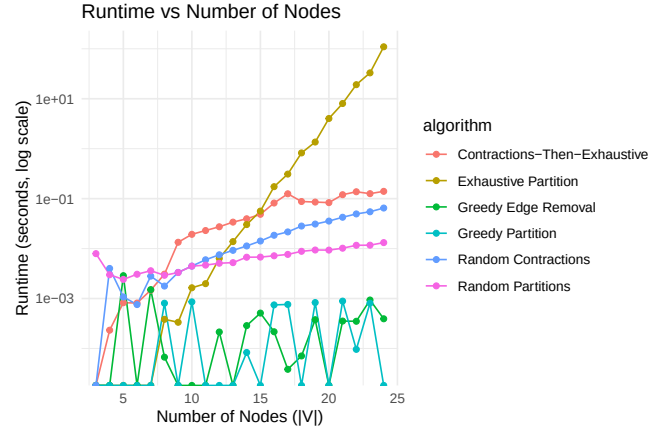


Fig. 5 - Logarithm of the running time in seconds in relation to the number of nodes in the graph

even exhibit nearly constant execution time, taking no more than 0.1 second for each graph. Overall, none algorithm never takes more than 1 second to find the solution, proving that the initially set time limit of 5 seconds was virtually useless.

D. Number of Solutions Tested

Table III presents the Pearson correlation between the number of solutions tested and the number of basic operations for each algorithm. We can see that for all algorithms the correlation is strong and positive, indicating that the number of solutions tested is a reliable proxy for the computational effort. In particular, the fully exhaustive method **Exhaustive Partition** shows nearly perfect correlation (0.999), while the random contraction methods have slightly lower correlation values, reflecting variability due to the randomized steps and non-dominant complexity members that are performed before testing solutions that have already been hashed. Surprisingly, **Random Partitions** show higher correlation than their ordinary counterpart, in spite of not counting the crossing edges on the already tested solutions and not trying to improve those solutions.

TABLE III
PEARSON CORRELATION BETWEEN NUMBER OF SOLUTIONS TESTED AND BASIC OPERATIONS FOR EACH ALGORITHM.

Algorithm	Pearson Correlation
Greedy Partition	0.797
Greedy Edge Removal	0.967
Random Partitions	0.846
Exhaustive Partition	0.999
Contraction-Then-Exhaustive	0.787
Random Contractions	0.651

E. Performance Analysis on Larger Graphs

Contractions-Then-Exhaustive. To test the per-

TABLE IV

SUMMARY OF MINIMUM CUT ALGORITHM PERFORMANCE. BEST RATE (%), AND ERROR STATISTICS ARE PRESENTED.

Algorithm	Best Rate (%)	Max Error	MAE
Greedy Edge Removal	100.0	0	0.00
Greedy Partition	97.8	3	0.85
Random Partitions	95.5	6	0.69
Random Contractions	51.5	65	8.12
Contractions-Then-Exhaustive	38.8	82	10.34

formance on the larger graphs, all of the algorithms were additionally tested on larger graphs, containing up to 500 vertices and with different densities. Table IV summarizes those results. As we can see, the best results are obtained by the heuristic algorithms that do not incorporate randomness, with **Greedy Edge Removal** always having the smallest found cut size. Both heuristic partitioning and its variant with random initializations achieve high match rates of more than 95%, while both methods that utilize random contractions achieve much lower match rates, especially

Figures 6 and 7 may provide the answer to why is that. While we can see that the number of operations grows sub-exponentially in relation to the number of edges, it is important to note that algorithms that utilize randomness have an upper bound of 5 seconds for finding the solution, therefore having an implicit upper bound on the number of operations that they can perform. However, it seems that the number of basic operations remains the same for **Greedy Partitions** and **Random Partitions** algorithms, despite the second one always running 100 iterations.

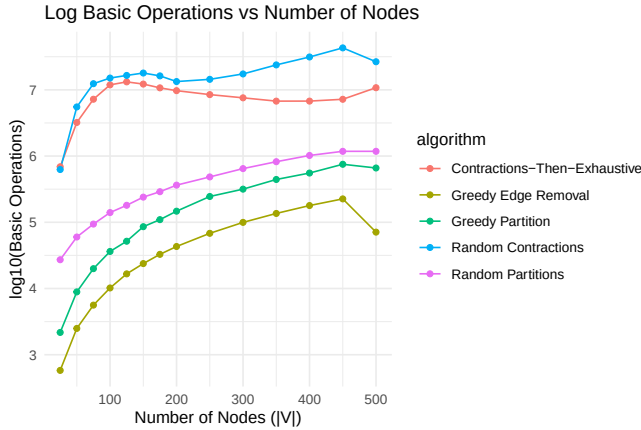


Fig. 6 - Logarithm of the average number of basic operations versus the number of vertices for the tested algorithms on larger graphs.

However, as shown on figure 7, the number of candidate solutions tested by both **Random Contractions** and **Random-Then-Exhaustive** decreases as we increase the graph, implicating that a number of basic operations conducted while testing a single candidate solution increases rapidly as we increase the graph size. Therefore, the algorithms implicitly decrease their potential for finding optimal solution by using the flat running time limit. On the other hand, the algorithm that incorporates greedy heuristic with multiple ran-

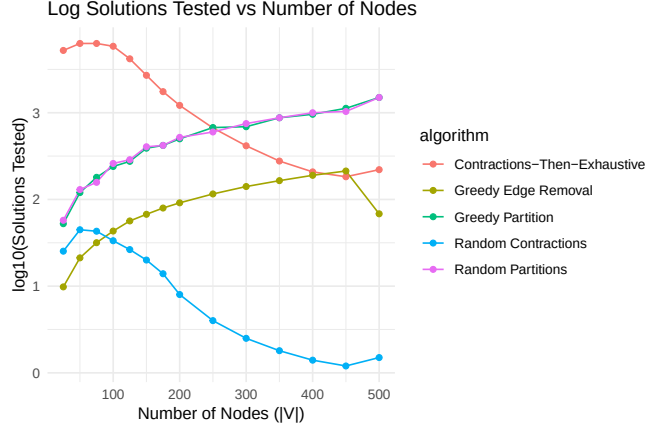


Fig. 7 - Logarithm of the average number of tested solutions versus the number of vertices for the tested algorithms on larger graphs.

dom initializations seems to have much larger solution space scope than the completely random algorithms, especially as the graph size increases. This confirms the previous hypothesis that it is much more expressive than the simple **Greedy Edge Removal**, although it seems that multiple independent random initializations do not increase significantly its scope of solution space.

V. DISCUSSION

The experimental results demonstrate that although random algorithms provide an interesting approach to exploring the solution space of problems that suffer from combinatorial explosion, their performance is highly dependent on the number of iterations and the inherent stochasticity of the process. While **Random Partitions** reduces the frequency of small-magnitude errors compared to purely greedy heuristics, its effectiveness diminishes on larger or more complex graphs due to fixed runtime constraints. Similarly, **Random Contractions** and **Contractions-Then-Exhaustive** exhibit lower best-found rates and higher mean absolute errors, indicating that random exploration alone cannot reliably guarantee near-optimal solutions in practical settings. Ideally, it should have some "guided search" for the optimal solution, instead of pure randomness.

In contrast, deterministic heuristics like **Greedy Edge Removal** and **Greedy Partitioning** consistently achieve high match rates with modest increase in computational cost. Greedy partitioning, in particular, balances local reasoning and global awareness by

evaluating vertex sets rather than individual vertices, allowing it to detect inter-cluster minimum cuts that greedy edge removal may miss. Moreover, hybrid approaches that combine greedy heuristics with randomized initializations also prove as a valuable alternative, increasing the accuracy on smaller graphs.

Overall, these findings emphasize the trade-off between solution quality, computational efficiency, and scalability: deterministic approaches achieve the best results in terms of precision and computational complexity, while purely random algorithms are dependent on the number of independent iterations and lack structural awareness of the graph, input on how do their decisions affect the graph structure.

VI. CONCLUSION

This paper evaluated several algorithms for solving the minimum cut in undirected, unweighted graphs, including exhaustive method, greedy heuristics, randomized and hybrid approaches. The experiments confirmed the expected exponential scaling of exhaustive methods, making them impractical for even moderately sized graphs, despite guaranteeing optimality.

In contrast, both greedy heuristics demonstrated substantially improved scalability, with polynomial or near-polynomial growth in computation and only minor losses in precision. Among these, Greedy Edge Removal was the fastest and generally most accurate, but its purely local decision-making limits its ability to detect cuts defined by higher-order graph structure. The Greedy Partitioning heuristic, although slightly slower, captures much more global structural information of the graph.

Randomized methods, including Random Edge Contractions and multiple random partition initializations combined with greedy approach exhibited variable success rates depending on the graph structure. While purely random contractions sometimes missed optimal cuts and achieved poor results, hybrid strategy showed robustness and solution quality comparable to pure heuristic methods. This highlights that controlled randomness can effectively expand the search space in comparison to purely random methods and complements well with heuristic methods.

The main limitation of this work is that once again all experiments were conducted only on synthetically generated graphs, and the applicability of these algorithms to real-world graph datasets remains to be checked. Therefore, future work should try to (1) enhance greedy partitioning and hybrid algorithms to avoid local optima, potentially via evolutionary strategies, and (2) test the developed algorithms on real-world graphs with known or verifiable minimum cuts.

Overall, the results reinforce that exhaustive approaches are intractable for larger instances, while carefully designed heuristic and randomized methods offer a practical, efficient alternative for solving the minimum cut problem on complex graphs.

REFERENCES

- [1] Marko Pongrac, “Design, analysis and experimental evaluation of exhaustive and greedy algorithms for the minimum cut problem”, Unpublished project report, University of Aveiro, 2025, Available from personal archive.
- [2] David E. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*, Addison-Wesley Longman Publishing Co., Inc., USA, 1st edition, 1989.
- [3] David R. Karger, “Global min-cuts in rnc, and other ramifications of a simple min-cut algorithm”, in *Proceedings of the Fourth Annual ACM-SIAM Symposium on Discrete Algorithms*, USA, 1993, SODA ’93, p. 21–30, Society for Industrial and Applied Mathematics.
- [4] Python Wiki contributors, “Timecomplexity”, <https://wiki.python.org/moin/TimeComplexity>, 2025, Accessed: 2025-12-01.
- [5] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart, “Exploring network structure, dynamics, and function using networkx”, in *Proceedings of the 7th Python in Science Conference (SciPy2008)*, Gäel Varoquaux, Travis Vaught, and Jarrod Millman, Eds., Pasadena, CA, USA, August 2008, pp. 11–15.
- [6] Python Software Foundation, *Python: A dynamic, open source programming language*, 2024.
URL: <https://www.python.org/>